

Code:

```
#include <stdio.h>

struct Process {

    int pid;

    int arrival_time;

    int burst_time;

    int deadline;

    int abs_deadline;

    int remaining_time;

    int is_completed;

    int completion_time;

    int turnaround_time;

    int waiting_time;

};

int main() {

    int n;

    printf("Enter the number of processes: ");

    scanf("%d", &n);


    struct Process p[n];

    float utilization = 0.0;

    float total_tat = 0, total_wt = 0;

    int total_burst = 0;


    for (int i = 0; i < n; i++) {

        p[i].pid = i + 1;

        printf("\nProcess %d\n", p[i].pid);

        printf("Enter Arrival Time: ");

        scanf("%d", &p[i].arrival_time);

        printf("Enter Burst Time (Ci): ");

        scanf("%d", &p[i].burst_time);

        printf("Enter Deadline/Period (Di): ");

        scanf("%d", &p[i].deadline);
```

```

    p[i].remaining_time = p[i].burst_time;
    p[i].abs_deadline = p[i].arrival_time + p[i].deadline;
    p[i].is_completed = 0;
    utilization += (float)p[i].burst_time / p[i].deadline;
    total_burst += p[i].burst_time;
}
printf("\n\n");
printf("CPU Utilization: %.4f\n", utilization);
if (utilization > 1.0) {
    printf("Result: Scheduling not feasible (deadlines may be missed).\n");
} else {
    printf("Result: Scheduling feasible.\n");
}
printf("\n");
int current_time = 0;
int completed_processes = 0;
int gantt_chart[200];
int time_log = 0;
while (completed_processes < n) {
    int min_deadline = 9999999;
    int selected_process_index = -1;
    for (int i = 0; i < n; i++) {
        if (p[i].arrival_time <= current_time && p[i].is_completed == 0) {
            if (p[i].abs_deadline < min_deadline) {
                min_deadline = p[i].abs_deadline;
                selected_process_index = i;
            }
        }
    }
    if (selected_process_index == -1) {
        gantt_chart[time_log++] = 0; // 0 represents Idle time
        current_time++;
    } else {

```

```

int i = selected_process_index;

gantt_chart[time_log++] = p[i].pid;

p[i].remaining_time--;

current_time++;

if (p[i].remaining_time == 0) {
    p[i].is_completed = 1;
    completed_processes++;
    p[i].completion_time = current_time;
    p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;
    p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;
    total_tat += p[i].turnaround_time;
    total_wt += p[i].waiting_time;
    if (p[i].completion_time > p[i].abs_deadline) {
        printf("\n[WARNING] Process %d missed its deadline! (Completed at %d, Deadline %d)\n",
            p[i].pid, p[i].completion_time, p[i].abs_deadline);
    }
}

}

}

printf("\n\n");

printf("PID\tArrival\tBurst\tRel. Dline\tAbs. Dline\tCompletion\tTurnaround\tWaiting\n");

printf("\n");

for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].arrival_time, p[i].burst_time, p[i].deadline,
        p[i].abs_deadline, p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
}

printf("\n");

printf("\nGANTT CHART\n");

printf(" ");

for (int i = 0; i < time_log; i++) printf("--- ");

printf("\n|");

for (int i = 0; i < time_log; i++) {
    if (gantt_chart[i] == 0) printf(" ID |");

```

```

        else printf(" P%d |", gantt_chart[i]);
    }

    printf("\n ");

    for (int i = 0; i < time_log; i++) printf("--- ");

    printf("\n0");

    for (int i = 0; i < time_log; i++) {

        if (i + 1 > 9) printf(" %d", i + 1);

        else printf(" %d", i + 1);

    }

    printf("\n\nAverage Turn Around Time: %.2f", total_tat / n);

    printf("\n\nAverage Waiting Time: %.2f\n", total_wt / n);

    return 0;
}

```

Output:

```

Enter the number of processes: 3

Process 1
Enter Arrival Time: 0
Enter Burst Time (Ci): 4
Enter Deadline/Period (Di): 7

Process 2
Enter Arrival Time: 2
Enter Burst Time (Ci): 5
Enter Deadline/Period (Di): 6

Process 3
Enter Arrival Time: 3
Enter Burst Time (Ci): 5
Enter Deadline/Period (Di): 7

-----
CPU Utilization: 2.1190
Result: Scheduling not feasible (deadlines may be missed).
-----

[WARNING] Process 2 missed its deadline! (Completed at 9, Deadline 8)

[WARNING] Process 3 missed its deadline! (Completed at 14, Deadline 10)

-----


| PID | Arrival | Burst | Rel. Dline | Abs. Dline | Completion | Turnaround | Waiting |
|-----|---------|-------|------------|------------|------------|------------|---------|
| 1   | 0       | 4     | 7          | 7          | 4          | 4          | 0       |
| 2   | 2       | 5     | 6          | 8          | 9          | 7          | 2       |
| 3   | 3       | 5     | 7          | 10         | 14         | 11         | 6       |


-----

GANTT CHART
-----
| P1 | P1 | P1 | P1 | P2 | P2 | P2 | P2 | P2 | P3 | P3 | P3 | P3 | P3 |
-----
0  1  2  3  4  5  6  7  8  9  10 11 12 13 14

Average Turn Around Time: 7.33
Average Waiting Time: 2.67
PS D:\vishwanath\OS LAB>

```

Proportional share scheduling

Code:

```
#include <stdio.h>

struct Process {
    int pid;
    float arrival_time;
    float burst_time;
    float weight;
    float remaining_time;
    int is_completed;
    float completion_time;
    float turnaround_time;
    float waiting_time;
};

struct Gantt {
    int pid;
    float start;
    float end;
};

int main() {
    int n;
    float time_quantum;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the Base Time Quantum: ");
    scanf("%f", &time_quantum);
    struct Process p[n];
    struct Gantt chart[100];
    int chart_count = 0;
    float total_weight = 0.0;
    float total_tat = 0.0, total_wt = 0.0;
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
```

```

printf("\nProcess %d\n", p[i].pid);
printf("Enter Arrival Time: ");
scanf("%f", &p[i].arrival_time);
printf("Enter Burst Time: ");
scanf("%f", &p[i].burst_time);
printf("Enter Weight: ");
scanf("%f", &p[i].weight);
p[i].remaining_time = p[i].burst_time;
p[i].is_completed = 0;
total_weight += p[i].weight;
}
float current_time = 0.0;
int completed_processes = 0;
while (completed_processes < n) {
    int executed_any = 0;
    for (int i = 0; i < n; i++) {
        if (p[i].arrival_time <= current_time && p[i].is_completed == 0) {
            if (total_weight <= 0) break;
            float time_slice = (p[i].weight / total_weight) * time_quantum;
            if (p[i].remaining_time < time_slice) {
                time_slice = p[i].remaining_time;
            }
            chart[chart_count].pid = p[i].pid;
            chart[chart_count].start = current_time;
            current_time += time_slice;
            p[i].remaining_time -= time_slice;
            chart[chart_count].end = current_time;
            chart_count++;
            executed_any = 1;
            if (p[i].remaining_time <= 0.0001) {
                p[i].remaining_time = 0;
                p[i].is_completed = 1;
                completed_processes++;
                p[i].completion_time = current_time;
            }
        }
    }
}

```

```

        p[i].turnaround_time = p[i].completion_time - p[i].arrival_time;

        p[i].waiting_time = p[i].turnaround_time - p[i].burst_time;

        total_tat += p[i].turnaround_time;

        total_wt += p[i].waiting_time;

        total_weight -= p[i].weight;
    }
}
}

if (executed_any == 0) {
    chart[chart_count].pid = 0;
    chart[chart_count].start = current_time;
    current_time += 1.0;
    chart[chart_count].end = current_time;
    chart_count++;
}
}

printf("\n-----\n");
printf("PID\tArrival\tBurst\tWeight\tCompletion\tTurnaround\tWaiting\n");
printf("-----\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\t%.2f\n",
        p[i].pid, p[i].arrival_time, p[i].burst_time, p[i].weight,
        p[i].completion_time, p[i].turnaround_time, p[i].waiting_time);
}

printf("-----\n");
printf("\n--- GANTT CHART ---\n\n");
for (int i = 0; i < chart_count; i++) printf("-----");
printf("\n|");
for (int i = 0; i < chart_count; i++) {
    if (chart[i].pid == 0) printf(" IDLE |");
    else printf(" P%d |", chart[i].pid);
}

printf("\n ");
for (int i = 0; i < chart_count; i++) printf("-----");

```

```

printf("\n0.00");

for (int i = 0; i < chart_count; i++) {
    printf(" %.2f", chart[i].end);
}

printf("\n\nAverage Turn Around Time: %.2f", total_tat / n);

printf("\nAverage Waiting Time: %.2f\n", total_wt / n);

return 0;
}

```

Output:

```

Enter the number of processes: 3
Enter the Base Time Quantum: 2

Process 1
Enter Arrival Time: 0
Enter Burst Time: 6
Enter Weight: 1

Process 2
Enter Arrival Time: 2
Enter Burst Time: 8
Enter Weight: 2

Process 3
Enter Arrival Time: 3
Enter Burst Time: 4
Enter Weight: 1

-----
PID   Arrival   Burst   Weight   Completion   Turnaround   Waiting
-----
1     0.00      6.00    1.00     18.00        18.00        12.00
2     2.00      8.00    2.00     17.00        15.00         7.00
3     3.00      4.00    1.00     17.50        14.50        10.50
-----

--- GANTT CHART ---

-----
| P1 | P1 | P1 | P1 | P2 | P3 | P1 | P2 | P3 | P1 | P2 | P3 | P1 | P2 | P3 | P1 | P2 | P3 | P1 | P2 | P3 | P1 | P2 | P3 | P1 |
-----
0.00  0.50  1.00  1.50  2.00  3.00  3.50  4.00  5.00  5.50  6.00  7.00  7.50  8.00  9.00  9.50 10.00 11.00 11.50 12.00 13.00 13.50 14.00 15.00 15.50 16.00 17.00 17.50 18.00

Average Turn Around Time: 15.83
Average Waiting Time: 9.83
PS D:\vishwanath\OS LAB>

```